COMS W4156 Advanced Software Engineering (ASE)

September 12, 2023

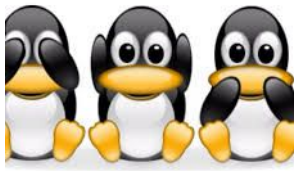
Agenda

1. Working in Teams
2. version control
3. coding style
4. code documentation
5. task board
6. Pair Programming
7. code review

Software Engineers always work in Teams

People who write run-once or short-lived programs are usually [programmers not software engineers]

Teams always have at least three members: you, the past you, and the future you



You / Past You / Future You

Motivate...

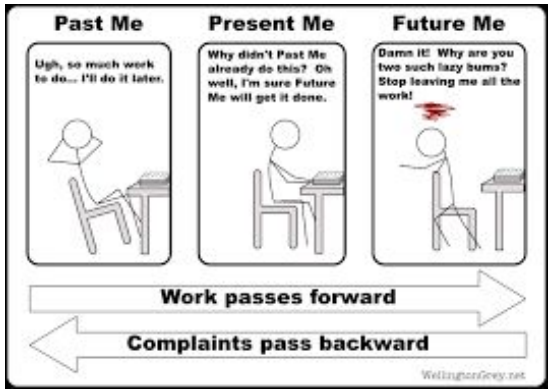
version control - It worked last week, can I undo my changes that broke it?

coding style - Why did I name this function ‘FixThisLater’ two years ago?

code documentation - What does ‘FixThisLater’ do?

task board - Where is the sticky note with my to-do list?

....even if your team never grows



Multi-Member Teams

There are usually more members, all of which join the team at some point (not necessarily beginning) and may leave at some point (not necessarily end)

A team of 4-5 software engineers often has >5 other members (some shared with other teams), such as tech lead, engineering manager, product manager, tech writer, customer support and/or devops liaison, UIUX designer and/or researcher (when applicable), possibly a separate testing team, with several layers of management above each

Some teams also have student interns or trainees

Multi-Member Engineering Teams

Mandate...

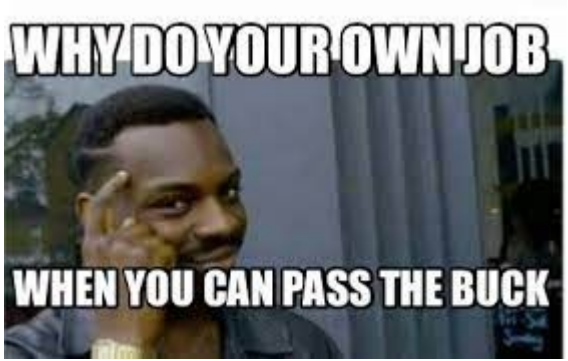
version control - My code worked last week, now it doesn’t, who broke it?

coding style - Why did Jing, who left two years ago to work for Kookle, name this function ‘FixThisLater’?

code documentation - What does ‘FixThisLater’ do?

task board - What am I supposed to be working on?

...particularly as team grows or churns

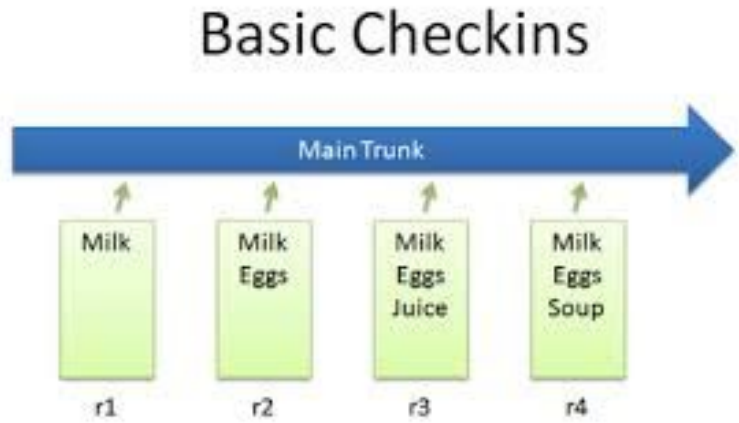


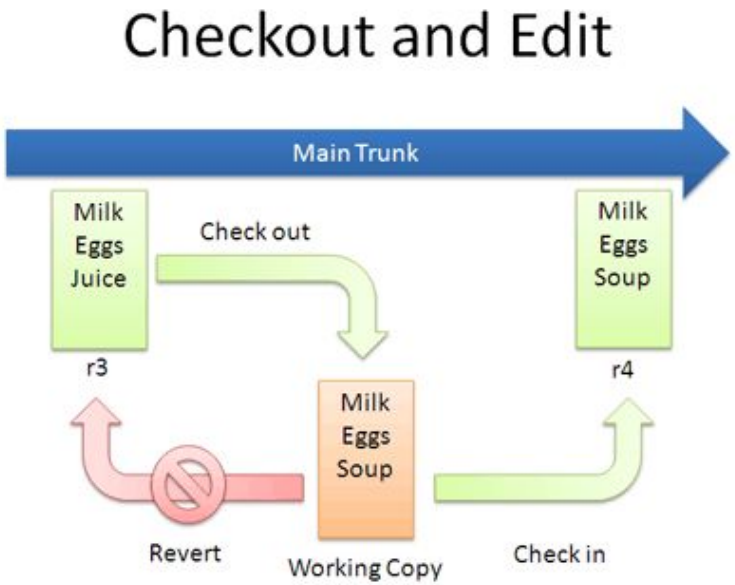
Agenda

1. Working in Teams
2. version control
3. coding style
4. code documentation
5. task board
6. Pair Programming
7. code review

Version Control

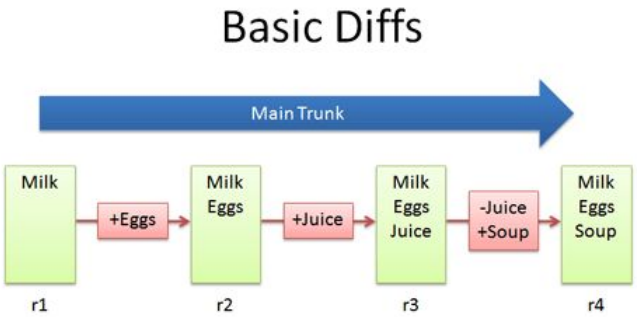
All of you should already understand [basic version control]

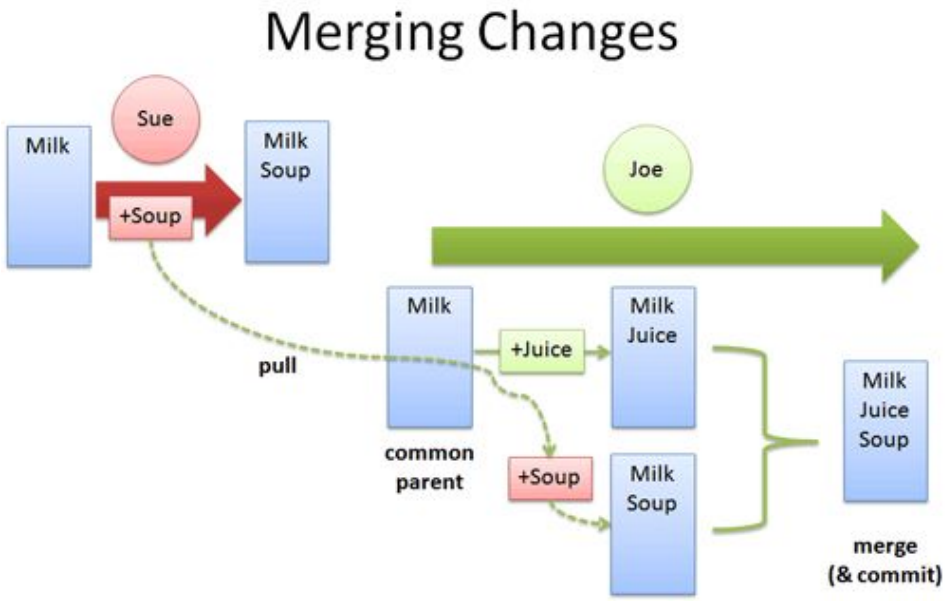




Version Control

But maybe not how merging changes works





Example Auto-Merge

```
int main(){
   char* ptr;
   FILE* data;
   char buf[100];
   ….some code….
   data = fopen("data.txt", "r");
   if(fgets(buf,100,data) == NULL)
      return 1;
   printf("Read: %s", buf);
   fclose(data);
   data = fopen("data.txt", "w");
   fwrite("juice\n", 6, 1, data);
   fclose(data);
   return 0;
   }
```

```
int main(){
   char* ptr;
   FILE* data;
   char buf[100];
   ….some code...
   data = fopen("data.txt", "r");
   if(fgets(buf,100,data) == NULL)
      return 1;
   printf("Read: %s", buf);
   fclose(data);
   data = fopen("data.txt", "w");
   fwrite("soup\n", 7, 1, data);
   fclose(data);
   return 0;
   }
```

What Could Go Wrong?

Example Needing Manual Resolution

```
/* original */
void display(char* str){
   write(1, str, strlen(str));
}
int main(){
   char* msg_ok = "Ok!\n";
   char* user = calloc(256, 1);
   char* pass = calloc(256, 1);
   strncpy(user, "default", 8);
   strncpy(pass, "default", 8);
   ...some code...
   write(2, msg_ok, strlen(msg_ok));
   display(user);
   return 0;
}
```

```
/* concurrent change */
void display(char* str){
   write(1, str, strlen(str));
}
int main(){
   char* msg_ok = "Ok!\n";
   char* user = calloc(256, 1);
   char* pass = calloc(256, 1);
   strncpy(user, "default", 8);
   strncpy(pass, "default", 8);
   …. some code...
   read(0, user, 255);
   read(0, pass, 255);
   write(2, msg_ok, strlen(msg_ok));
   display(user);
   return 0;
}
```

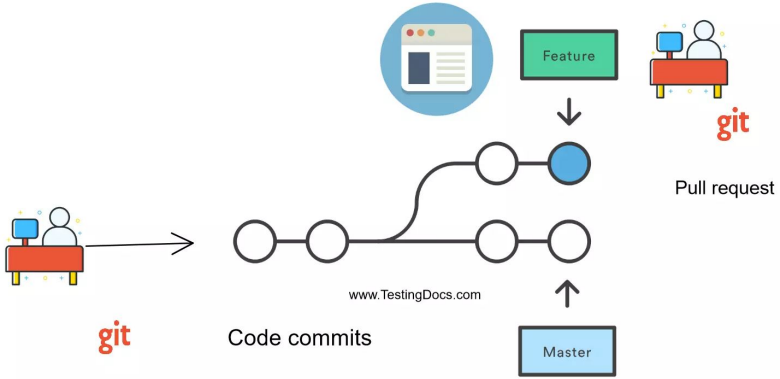
```
/* concurrent change */
void display(char* str){
   write(1, str, strlen(str));
}
int main(){
   char* msg_ok = "Ok!\n";
   char* user = calloc(256, 1);
   char* passwd = calloc(256, 1);
   strncpy(user, "default", 8);
   strncpy(passwd, "default", 8);
   ...some code...
   read(0, passwd, 255);
   write(2, msg_ok, strlen(msg_ok));
   display(user);
   return 0;
}
```

Why?

Version Control for Your Projects

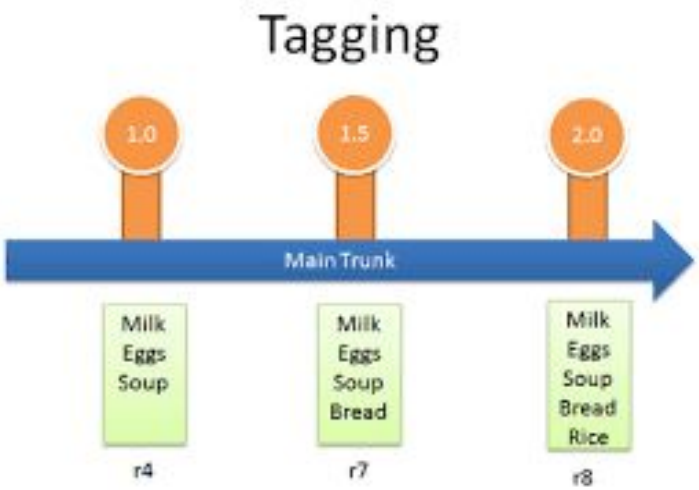
You are required to use github for your team projects

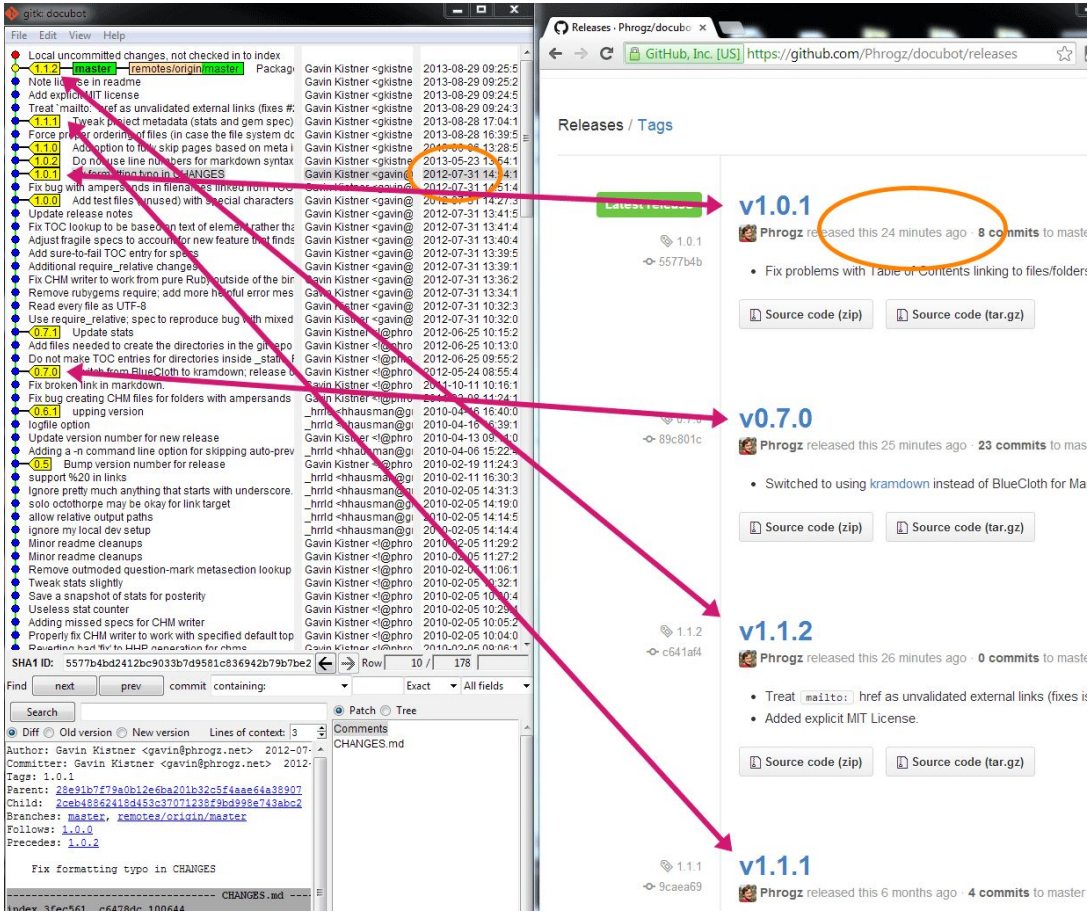
Use branches and pull requests. Do not do everything in the main branch



Release Tags

When you submit an iteration or present a demo, **tag** the revisions that contributed to that milestone so your mentor can easily find them





Agenda

1. Working in Teams
2. version control
3. coding style
4. code documentation
5. task board
6. Pair Programming
7. code review

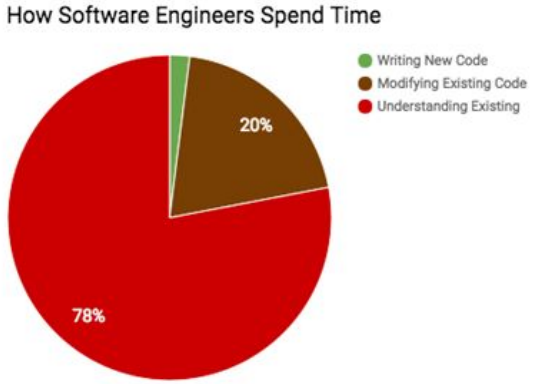
Coding Style

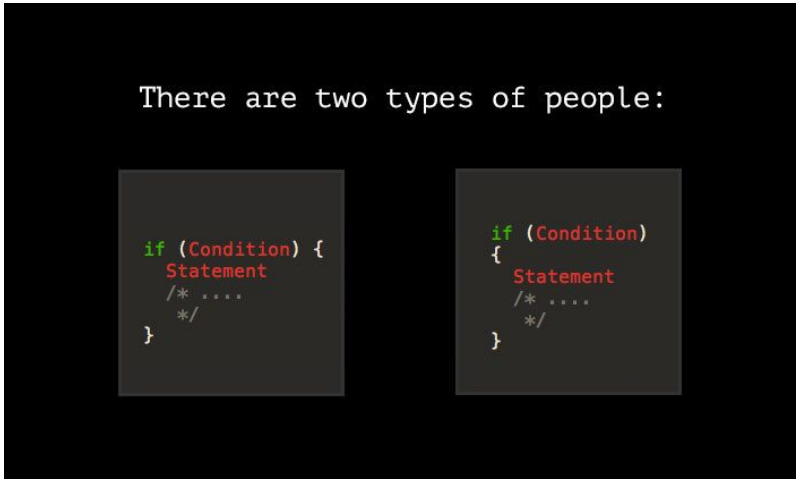
All (or nearly all) of you learned some standard coding style when you learned to program in xxx

Governs how and when to use comments and whitespace (indentation, blank lines), proper naming of variables and functions (e.g., camelCase, snake_case), code grouping and organization, file/folder structure

Compliance with a team's coding style helps other developers understand your code and vice versa, even helps your future self understand your past self's code

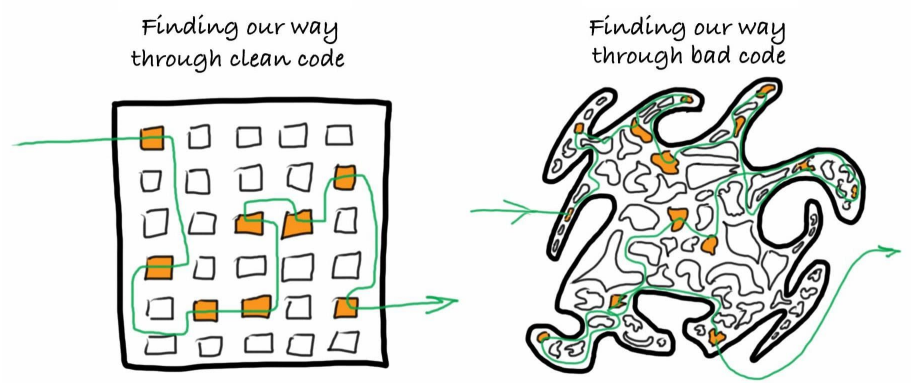
See Google's [many style guides]





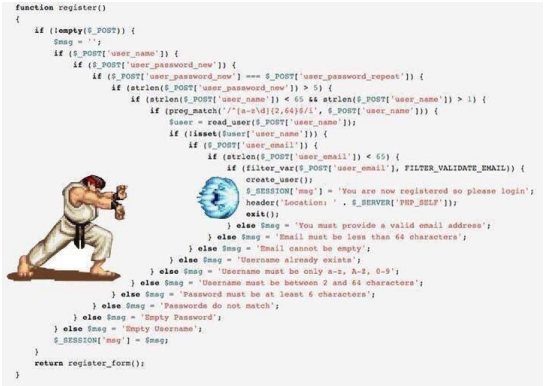
What is Wrong with this Code?

```
switch(value) {
    case 1:
        doSomething();

    case 2:
        doSomethingElse();
        break;

    default:
        doDefaultThing();
}
```



Which Tests Pass - neither, first, second, both?

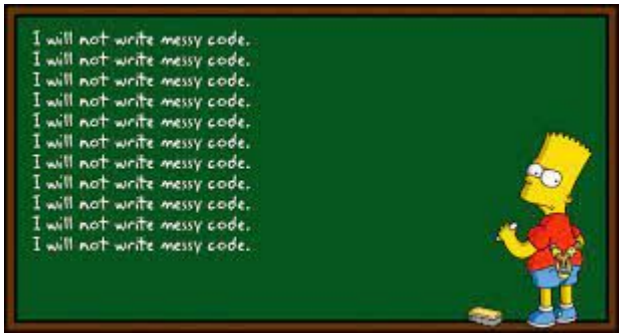
```java
public class TestBadCode {

    public int calculateFoo(int x, int y, boolean increment){
        if(increment)
            x++;
            x *=2;
        x += y;
        return x;
    }

    @Test
    public void test() {
        assertEquals(calculateFoo(3, 5, true), 13);
        assertEquals(calculateFoo(3, 5, false), 8);
    }

}
```



What Does This Code Do?

```
#include <stdio.h>

#define N(a)        "%"#a"$hhn"
#define O(a,b)      "%10$"#a"d"N(b)
#define U           "%10$.*37$d"
#define G(a)        "%"#a"$s"
#define H(a,b)      G(a)G(b)
#define T(a)        a a
#define s(a)        T(a)T(a)
#define A(a)        s(a)T(a)a
#define n(a)        A(a)a
#define D(a)        n(a)A(a)
#define C(a)        D(a)a
#define R           C(C(N(12)G(12)))
#define o(a,b,c)    C(H(a,a))D(G(a))C(H(b,b)G(b))n(G(b))O(32,c)R
#define SS          O(78,55)R "\n\033[2J\n%26$s";
#define E(a,b,c,d) H(a,b)G(c)O(253,11)R G(11)O(255,11)R H(11,d)N(d)O(253,35)R
#define S(a,b)      O(254,11)H(a,b)N(68)R G(68)O(255,68)N(12)H(12,68)G(67)N(67)

char* fmt = O(10,39)N(40)N(41)N(42)N(43)N(66)N(69)N(24)O(22,65)O(5,70)O(8,44)N(
            45)N(46)N    (47)N(48)N(    49)N( 50)N(     51)N(52)N(53    )O( 28,
            54)O(5,        55) O(2,    56)O(3,57)O(      4,58 )O(13,    73)O(4,
            71 )N(   72)O   (20,59    )N(60)N(61)N(       62)N (63)N    (64)R R
            E(1,2,   3,13   )E(4,    5,6,13)E(7,8,9        ,13)E(1,4    ,7,13)E
            (2,5,8,        13)E(    3,6,9,13)E(1,5,         9,13)E(3    ,5,7,13
            )E(14,15,    16,23)    E(17,18,19,23)E(          20, 21,    22,23)E
            (14,17,20,23)E(15,    18,21,23)E(16,19,    22     ,23)E(    14, 18,
            22,23)E(16,18,20,    23)R U O(255 ,38)R    G (     38)O(    255,36)
            R H(13,23)O(255,    11)R H(11,36) O(254    ,36)     R G(    36 ) O(
            255,36)R S(1,14    )S(2,15)S(3, 16)S(4,    17 )S     (5,    18)S(6,
            19)S(7,20)S(8,    21)S(9    ,22)H(13,23    )H(36,     67    )N(11)R
            G(11)""O(255,    25 )R        s(C(G(11)    ))n (G(          11) )G(
            11)N(54)R C(    "aa")   s(A(   G(25)))T    (G(25))N         (69)R o
            (14,1,26)o(    15, 2,   27)o   (16,3,28    )o( 17,4,        29)o(18
            ,5,30)o(19    ,6,31)o(        20,7,32)o    (21,8,33)o       (22 ,9,
            34)n(C(U)    )N( 68)R H(    36,13)G(23)    N(11)R C(D(      G(11)))
            D(G(11))G(68)N(68)R G(68)O(49,35)R H(13,23)G(67)N(11)R C(H(11,11)G(
            11))A(G(11))C(H(36,36)G(36))s(G(36))O(32,58)R C(D(G(36)))A(G(36))SS

#define arg d+6,d+8,d+10,d+12,d+14,d+16,d+18,d+20,d+22,0,d+46,d+52,d+48,d+24,d\
            +26,d+28,d+30,d+32,d+34,d+36,d+38,d+40,d+50,(scanf(d+126,d+4),d+(6\
            -2)+18*(1-d[2]%2)+d[4]*2),d,d+66,d+68,d+70, d+78,d+80,d+82,d+90,d+\
            92,d+94,d+97,d+54,d[2],d+2,d+71,d+77,d+83,d+89,d+95,d+72,d+73,d+74\
            ,d+75,d+76,d+84,d+85,d+86,d+87,d+88,d+100,d+101,d+96,d+102,d+99,d+\
            67,d+69,d+79,d+81,d+91,d+93,d+98,d+103,d+58,d+60,d+98,d+126,d+127,\
            d+128,d+129

char d[538] = {1,0,10,0,10};

int main() {
    while(*d) printf(fmt, arg);
}
```

Style Checkers



Most languages have automated tools that detect deviations from a prescribed coding style, | e.g., checkstyle or solarlint

The auto-merge feature of git and other version control systems depend on style compliance to avoid superficial, format-related merge conflicts - so developers should always run style checking and fix any problems *before* commiting code changes

Running the style checker (and code review tools, static analysis bug finders, unit tests) prior to each commit can be automated for github with a git pre-commit hook

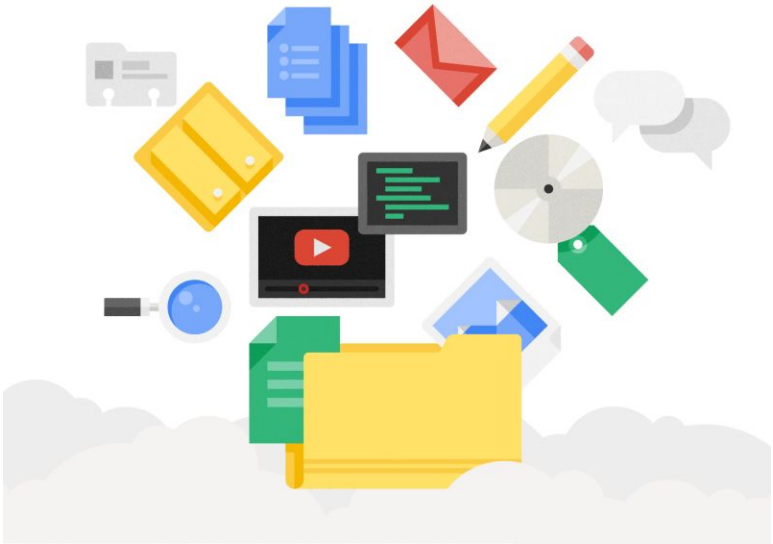
Style Checking for Your Projects

Each team is required to choose some reasonable coding style to be used by everyone on the team

Make sure it has a usable and actively maintained style checking tool that detects and reports any deviations from your chosen style

Agenda

1. Working in Teams
2. version control
3. coding style
4. code documentation
5. task board
6. Pair Programming
7. code review



Code Documentation at Unit Level

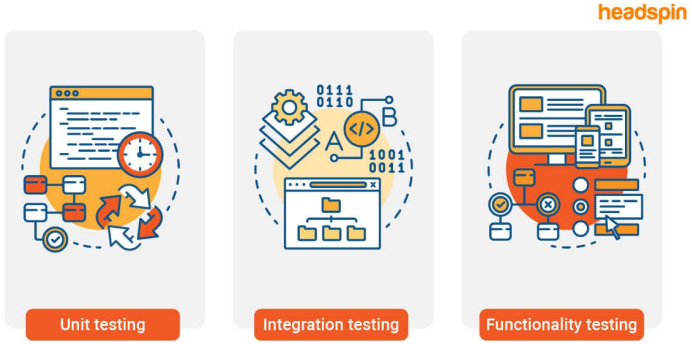
See Google's Documentation Best Practices and Engineering Practices
- "*Documentation is the story of your code*"

A non-trivial method (function, procedure, subroutine, etc.) has *inline* comments that explain "why" the code is written this way or "why" it works this way - intended audience is future developers who need to understand or modify this code

Methods also have a header (Javadoc, docstring, etc.) that explains "what" the method does, "how" to use it and how not to use it - intended audience is future developers who use this code in their own code in the same or a different class

Unit Tests as Documentation

Every behavior documented in a method header has several test cases to verify that behavior



These test cases themselves are documented with comments that explain:

- what environment or context needs to be set up before calling the method (and possibly torn down after it returns),
- what arguments the method takes,
- what values the method returns (if any),
- what non-local data the method reads and writes,
- what exceptions the method can throw or errors it can return,
- and any “gotchas” or restrictions on when and where to use this method

Code Documentation at Higher Level

Classes (modules, files, etc.) document the class as a whole, overview what the class does and show examples of how it might be used (both simple and advanced, if applicable) - intended audience is future developers who use or modify

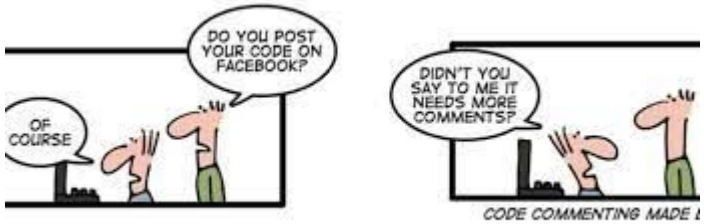
Folders (directories) have a README.md file that states What is this directory intended to hold? Which files should the developer look at first? Do some files define or implement an API? Who maintains this directory and where can I learn more?

There are even “prose-linters” like Vale that check some documentation content - different from spelling/grammar checkers, because they know they are checking code documentation (and can deal with code syntax in the midst of prose)

Code Documentation for Your Projects

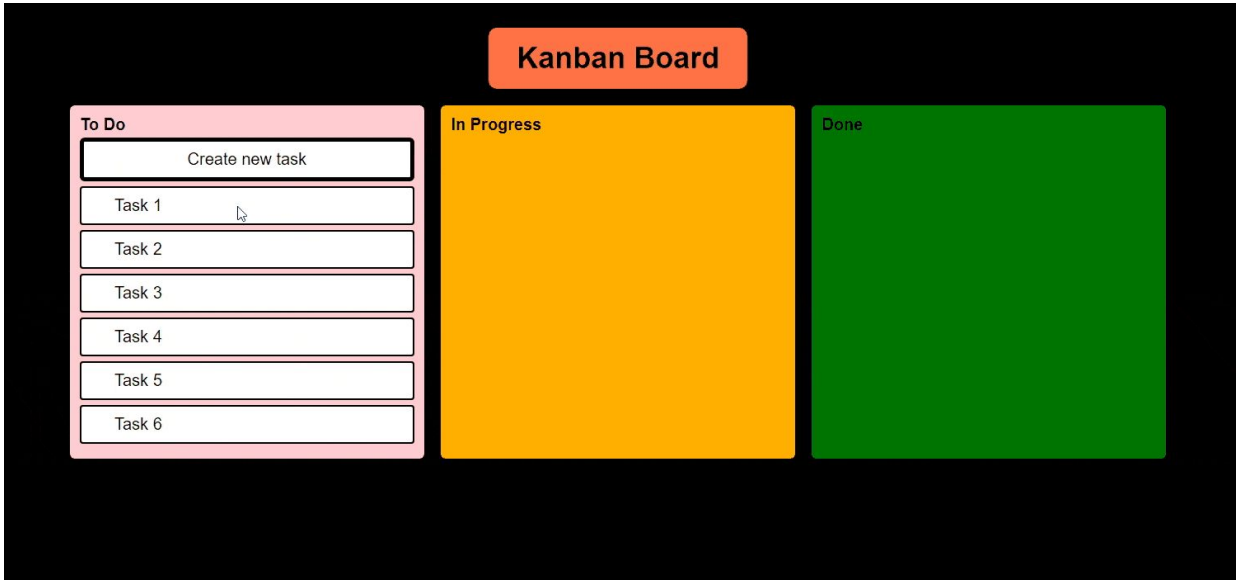
Document your code!

Your team mentor will check for documentation and try to read it



Agenda

1. Working in Teams
2. version control
3. coding style
4. code documentation
5. task board
6. Pair Programming
7. code review



Task Boards

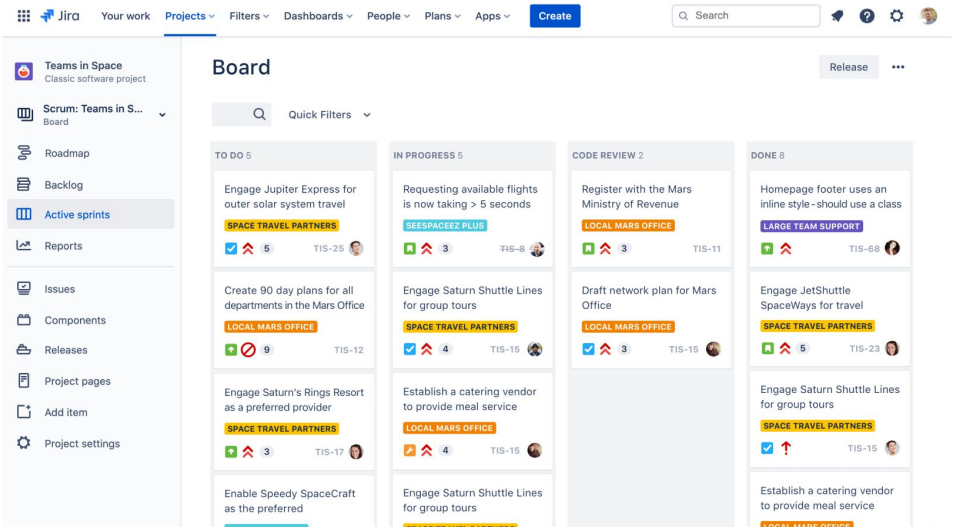
Documents who is working on what this iteration or sprint or week or …

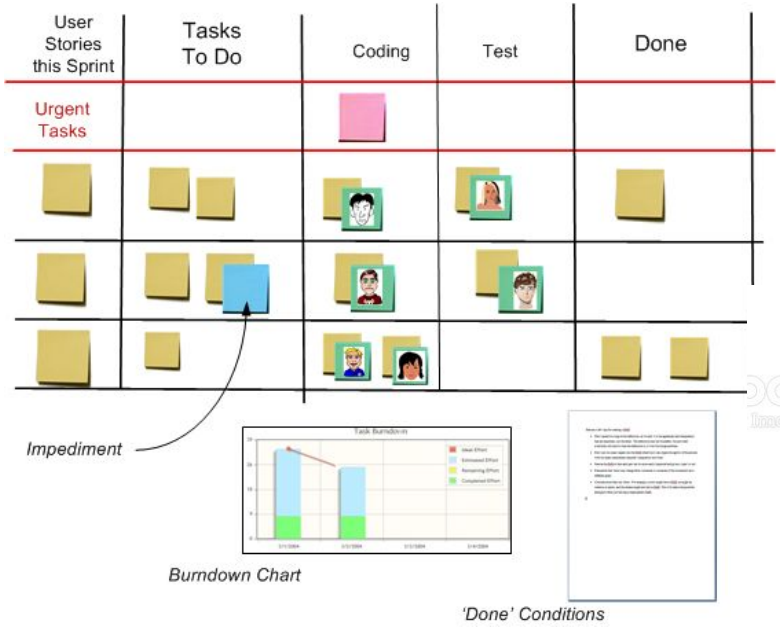
Task boards show to-do, in-progress, etc. columns of tickets - each with title, priority, who assigned to, tags, description, and links to other tickets, revisions and/or releases

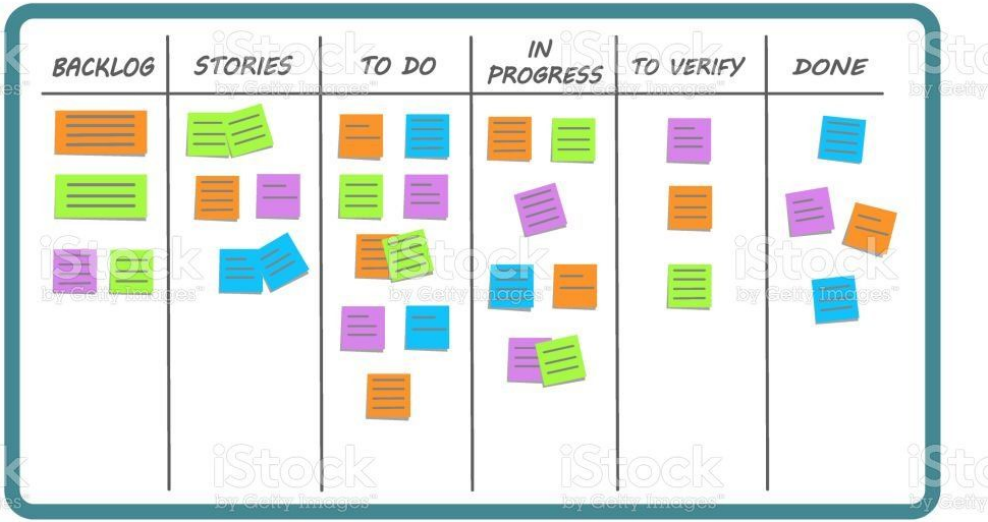
Tickets typically written and assigned by team leader or product owner

JIRA and Trello probably best-known systems, there's free versions for small teams

Github has built-in "issues" and "projects"







We strongly recommend, but do not require, that your team use a task board or issue tracking tool

Agenda

1. Working in Teams
2. version control
3. coding style
4. code documentation
5. task board
6. Pair Programming
7. code review

Multi-Member Engineering Teams

Enable…

pair programming!

Ought to be called “pair software engineering”,
but “pair programming” sounds better

Used for many software engineering activities, not just programming

Some companies use pair-programming interviews between job candidate and evaluator (current employee) even if their employees do not pair-program routinely

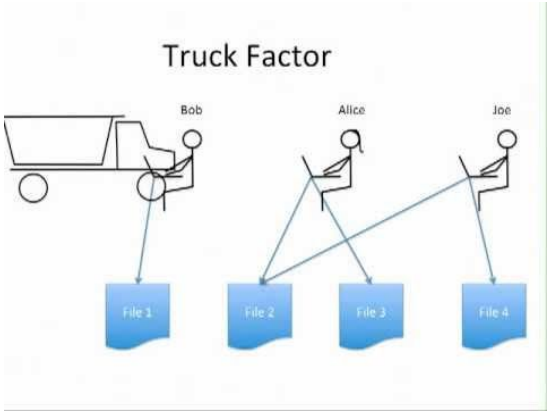
Ideal Pair Programming

Two people sit side by side at same computer or use remote desktop sharing

- Take turns “driving” (typing) vs. “navigating” (continuous review of code or whatever working on)
- Switch roles at frequent intervals (e.g., 25 minutes)
- The pair does **not** divide up the work, they try to do everything **together**
- If necessary to work separately on something, e.g., when one partner is absent, they review together

Benefits of Pair Programming

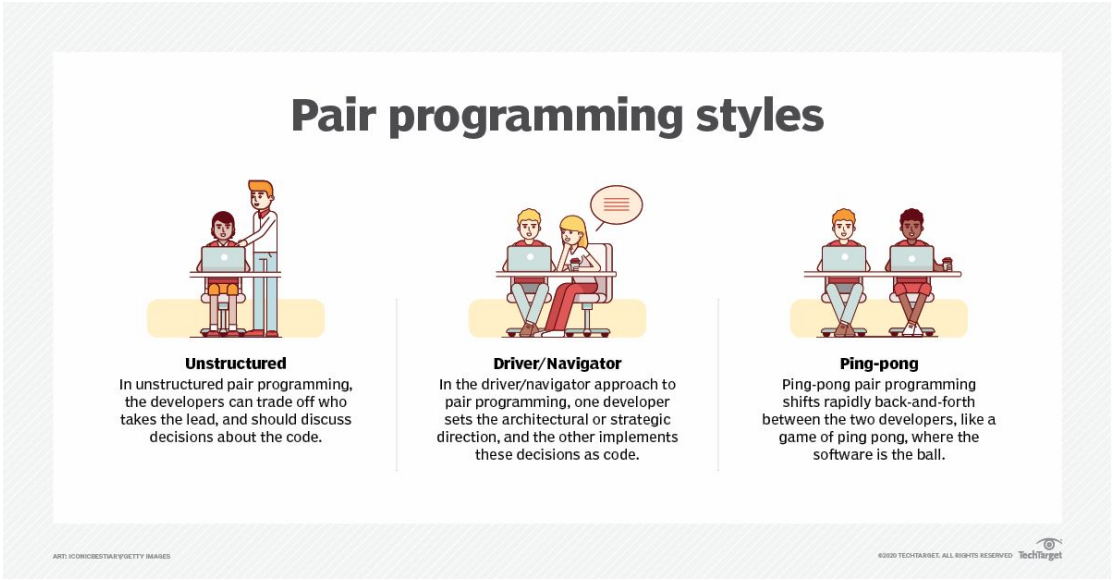
Reduce risk through collective code ownership and maximize "truck factor" = number of engineers that would have to disappear before project would be in serious jeopardy



- Shared responsibility to complete tasks on time
- Stay focused and on task
- Less likely to read email, surf web, etc.
- Less likely to be interrupted
- Partners expect "best practices" from each other
- Two people can solve problems that one couldn't do alone and/or produce better solutions
- Pool knowledge resources, improve skills

Pair Programming

Instant code review happens on the fly during pair programming, where the navigator reviews while the driver writes/edits



Training

Pair programming helps train junior developers - over-the-shoulder synchronous code review by senior developer

Style checkers, testing and other automated tools cannot find all bad code

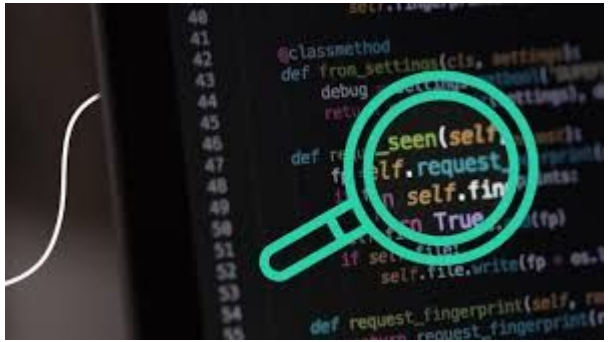
In “code review”, one or more *humans* read and analyze the code

Motivation for Code Review

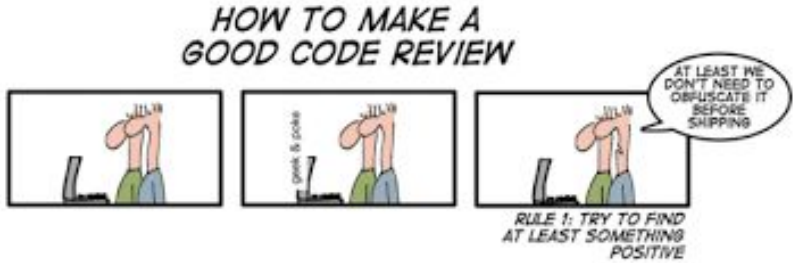
Like automated static analysis, code review can potentially consider *all* inputs whereas testing can only consider *some* inputs - often a very small subset of all inputs.

Code review can find problems that testing and static analyzers cannot, such as code that passes style checking and other static analyses, and passes all test cases, but is still unreadable

Humans reviewers can also spot application-specific bugs as well as generic bugs that are beyond the state of the art in static analysis, particularly important when running the code is expensive



Lightweight Code Review



Many organizations require code to be reviewed by a peer, a senior developer, or a designated reader prior to committing to the shared code repository

Lightweight forms of code review involve one or two readers besides the coder

May be synchronous (reviewer sits with coder) or asynchronous (separately)

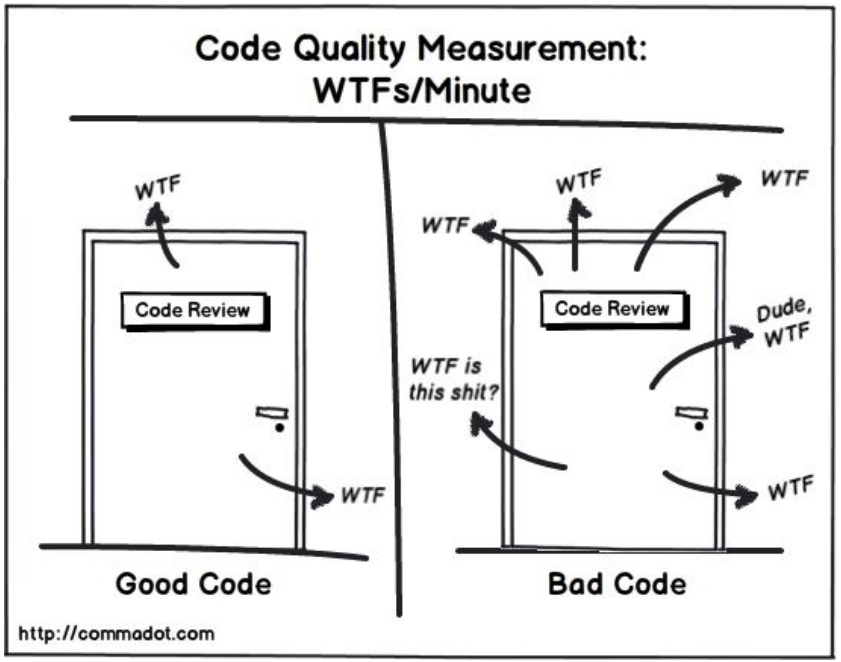
Aside from pair programming and junior developer training, there are major problems with synchronous reviews, so asynchronous generally considered better - submit prospective changes (pre-commit) for review and do something else until response

Formal Code Review

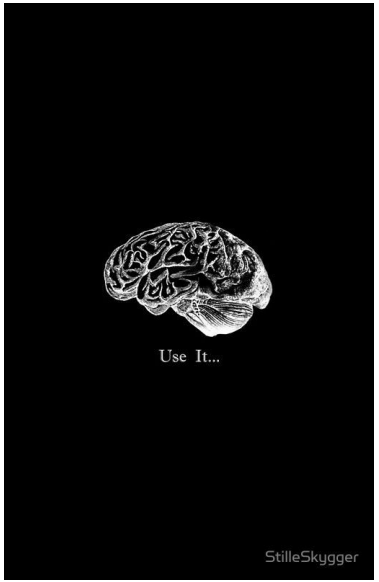
Formal code review (sometimes called Fagan inspection) is heavyweight - involves an elaborate *team* review process

Mandated for safety-critical software, or any software that must work right the first time and every time

Rarely used for conventional business and consumer software other than for training purposes



Pair Programming and Code Review for Your Projects



We strongly recommend pair programming for your team projects. This is why four-member teams are preferred to five-member teams, since triplet programming does not work very well

We strongly recommend lightweight code review for your team projects. Please ask some other member of your team to read your code (or your pair's code) before committing to the main trunk in your github repository, even if it passes all tests - this is the basis for "pull requests"

Upcoming Assignments

- ➔ Team Formation due next Monday, September 18, by 11:59pm
- ➔ Team Preliminary Project Proposal due Monday, September 25, by 11:59pm

Thursday

"Looking for Group"

Finding Bugs

Ask Me Anything